



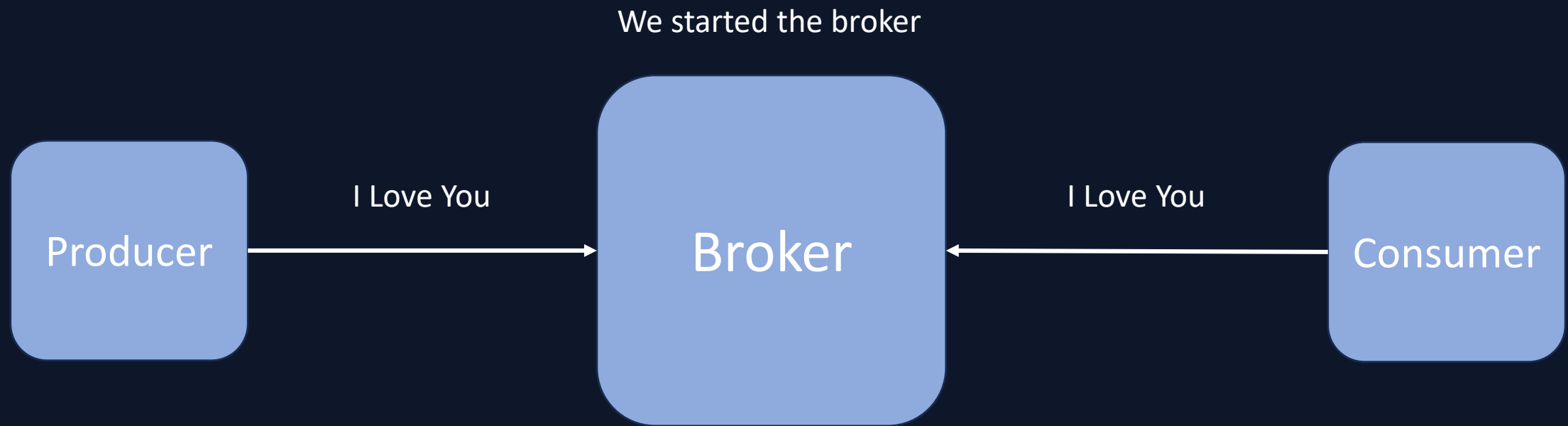
# Clustering in Apache Kafka?

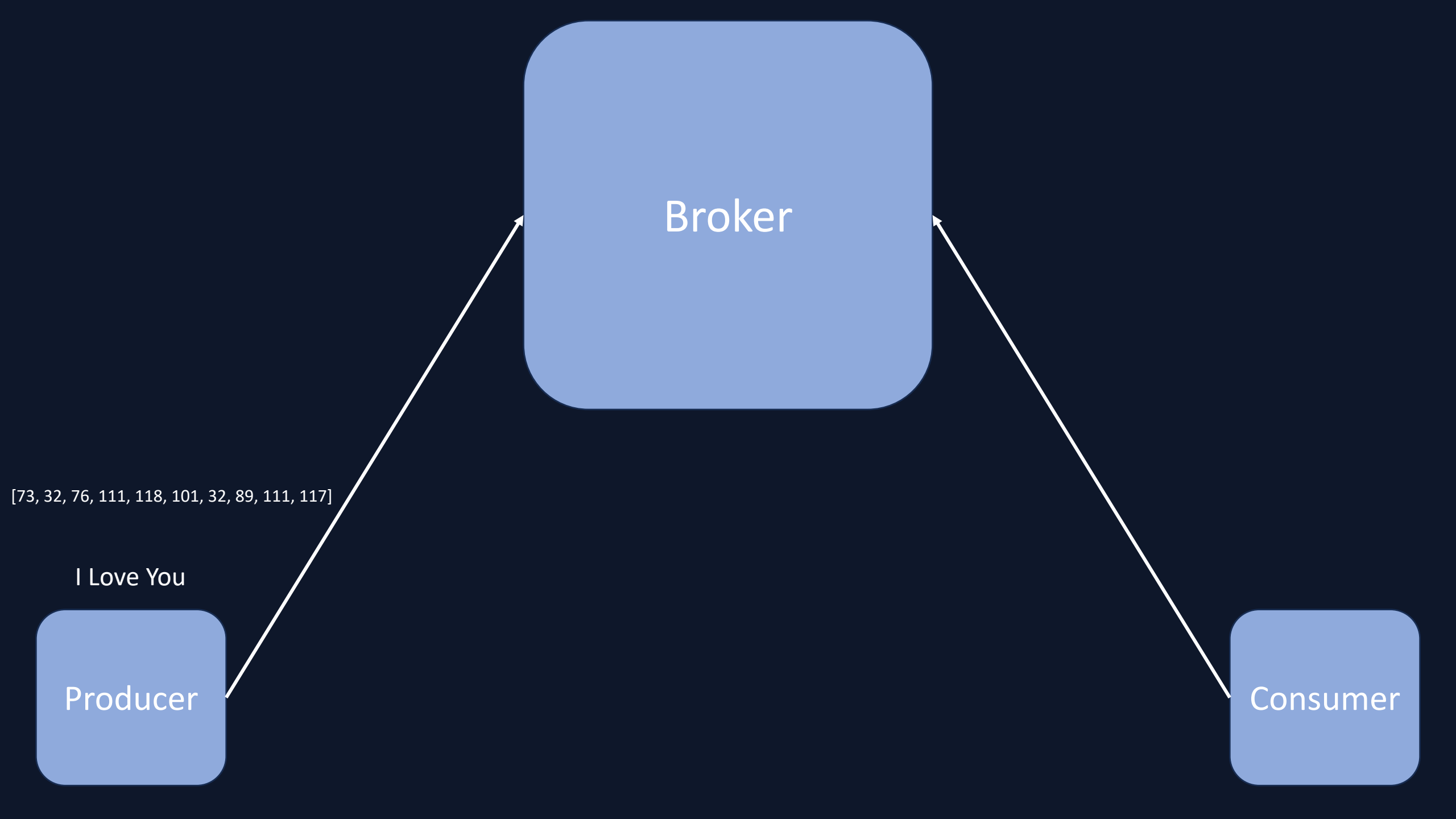
Mohammad Fozouni (Ph.D.)

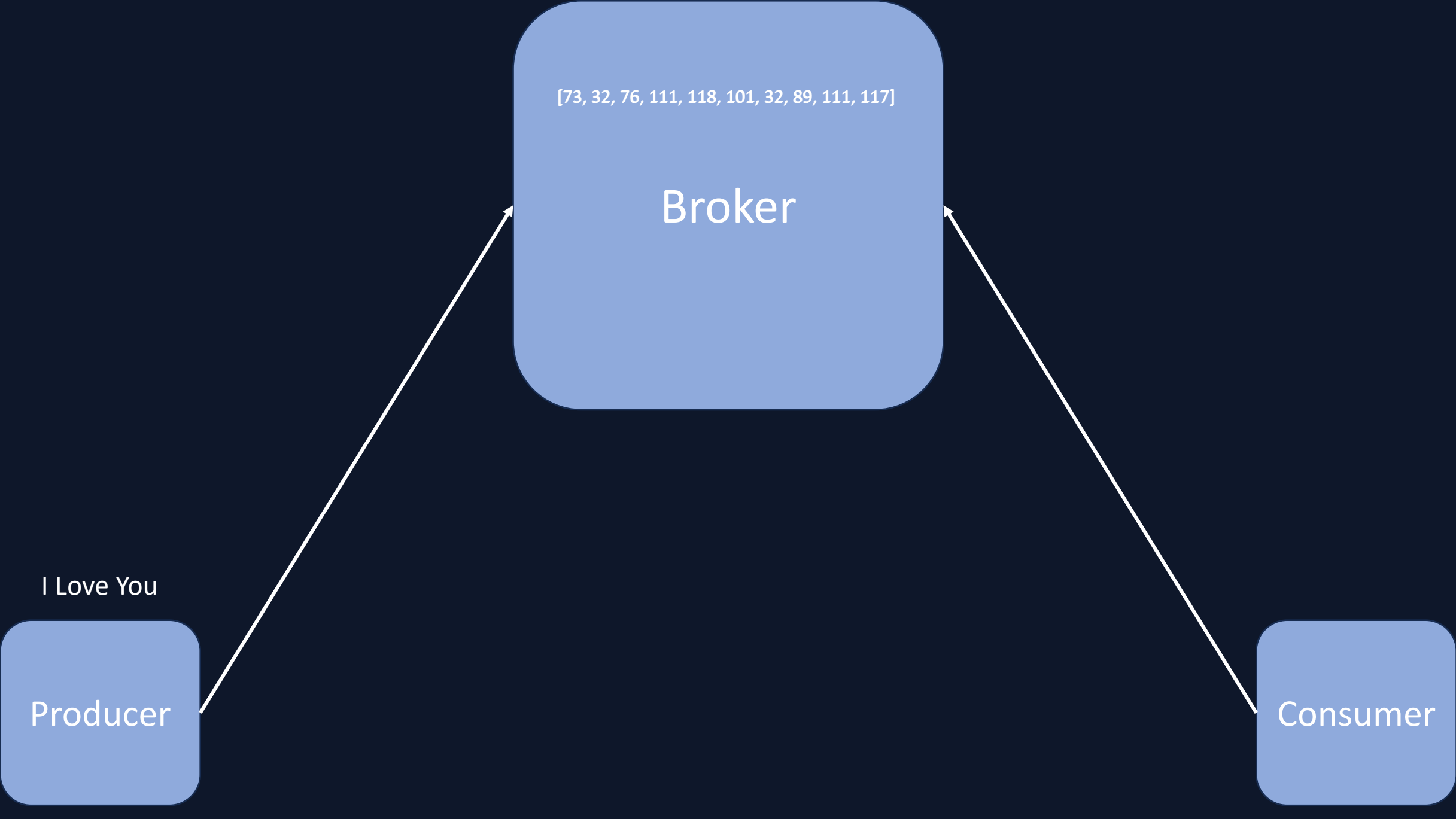
MLSECOPS IN ACTION COURSE

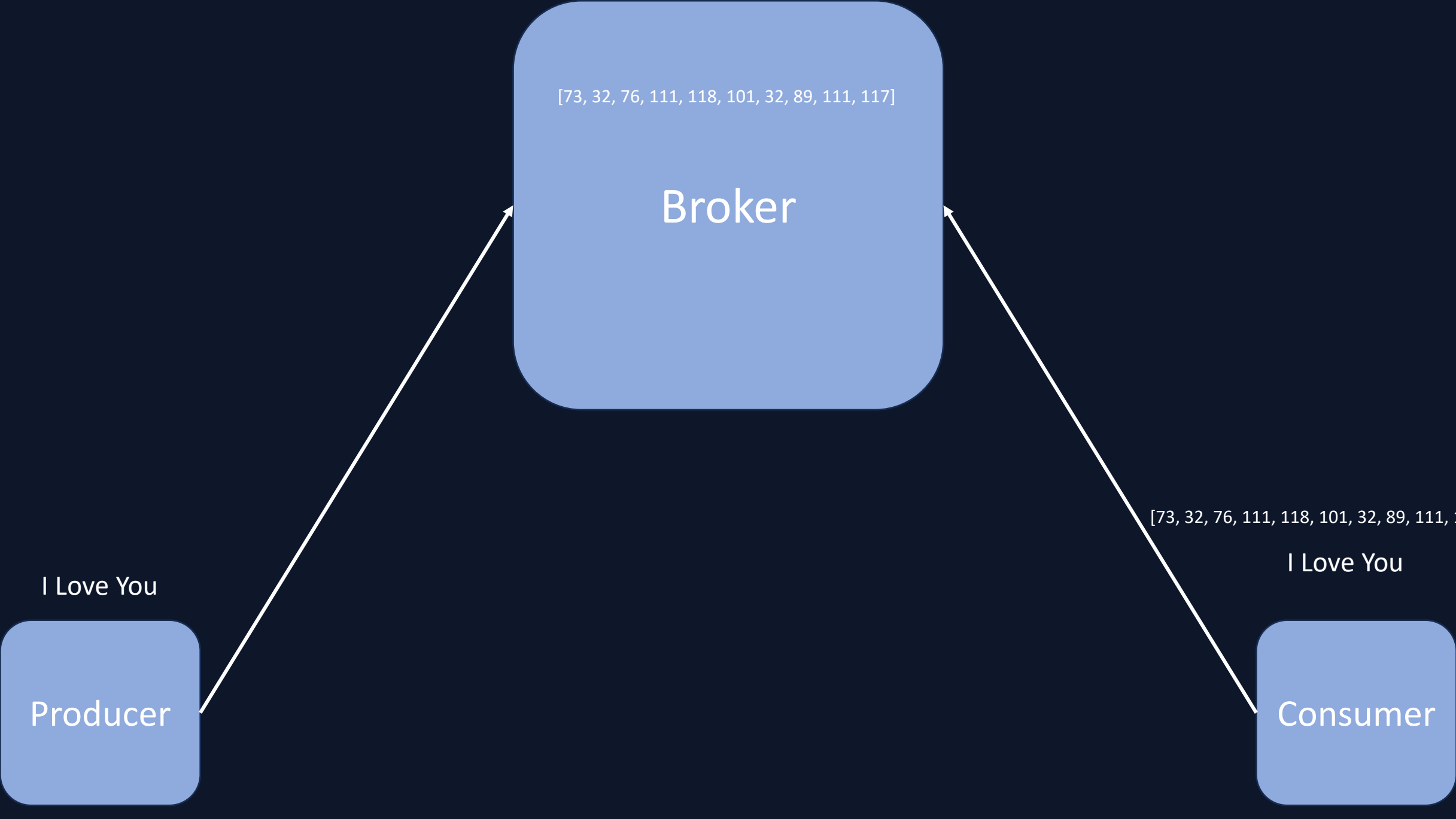


**What we saw in the  
previous session?**









# Why Clustering?

# Why Clustering?

بطور کلی فرایند کلاسترینگ  
در درک سیستم‌های توزیع شده  
بسیار مفید است. حتی انجام آن در  
سیستم لوکال (شخصی)



# Why Clustering?

بطور کلی فرایند کلاسترینگ  
در درک سیستم‌های توزیع شده  
بسیار مفید است. حتی انجام آن در سیستم لوکال (شخصی)

هنگامی که یک کلاستر از کافکا را تنظیم و آماده می‌کنیم،  
نحوه‌ی توزیع پیام‌ها بین پارتیشن‌ها را بشکل عملی خواهیم  
دید.

# Why Clustering?

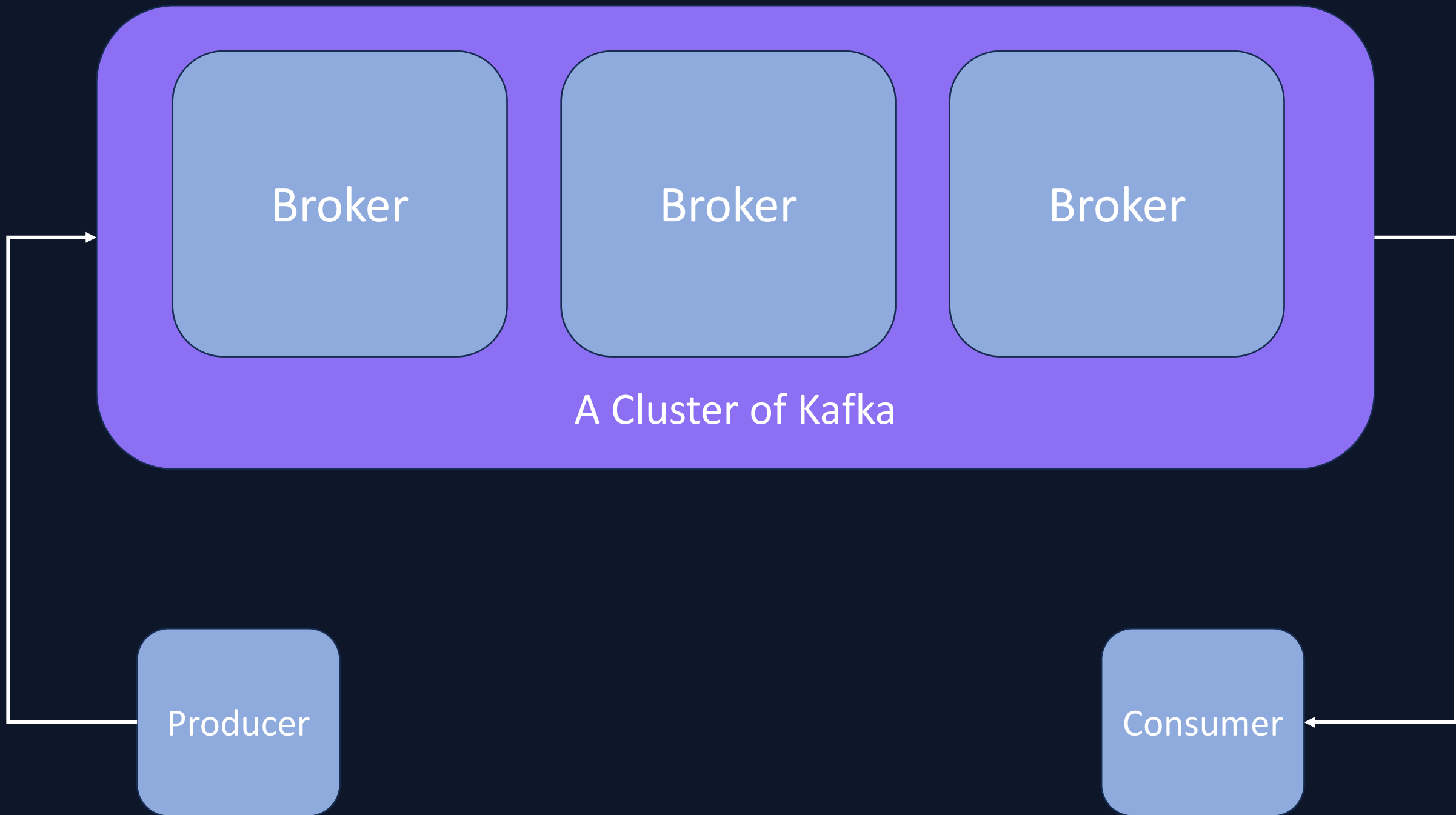
بطور کلی فرایند کلاسترینگ  
در درک سیستم‌های توزیع شده  
بسیار مفید است. حتی انجام آن در سیستم لوکال (شخصی)

هنگامی که یک کلاستر از کافکا را تنظیم و آماده می‌کنیم، نحوه‌ی توزیع پیام‌ها  
پارتیشن‌ها را بشکل عملی خواهیم دید.

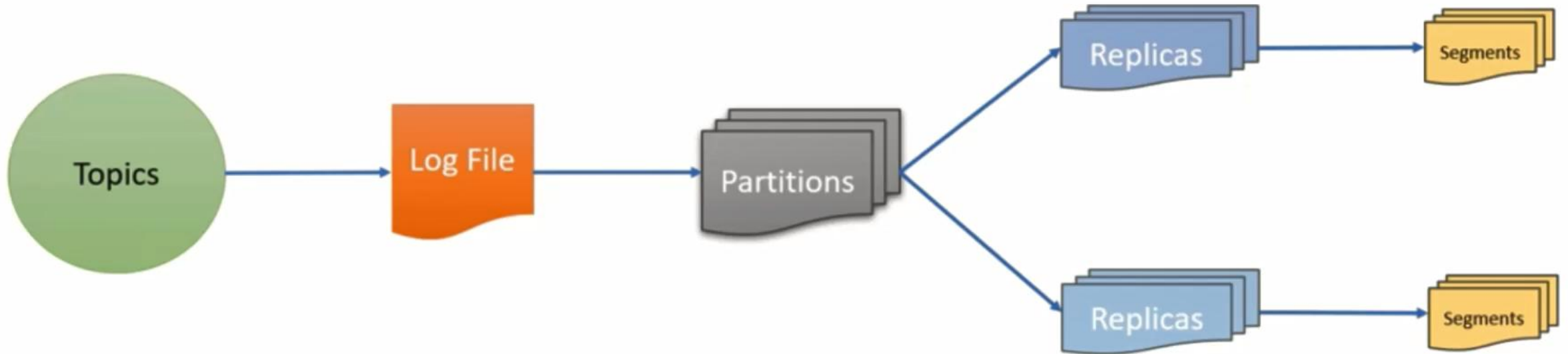
از لحاظ امنیتی نیز، هر چقدر رابطه بین اجزای مختلف کافکا در این حالت را بهتر درک  
کنیم، دقیق‌تر می‌توانیم فرایند هاردنینگ این مولفه در استک خود را انجام دهیم.

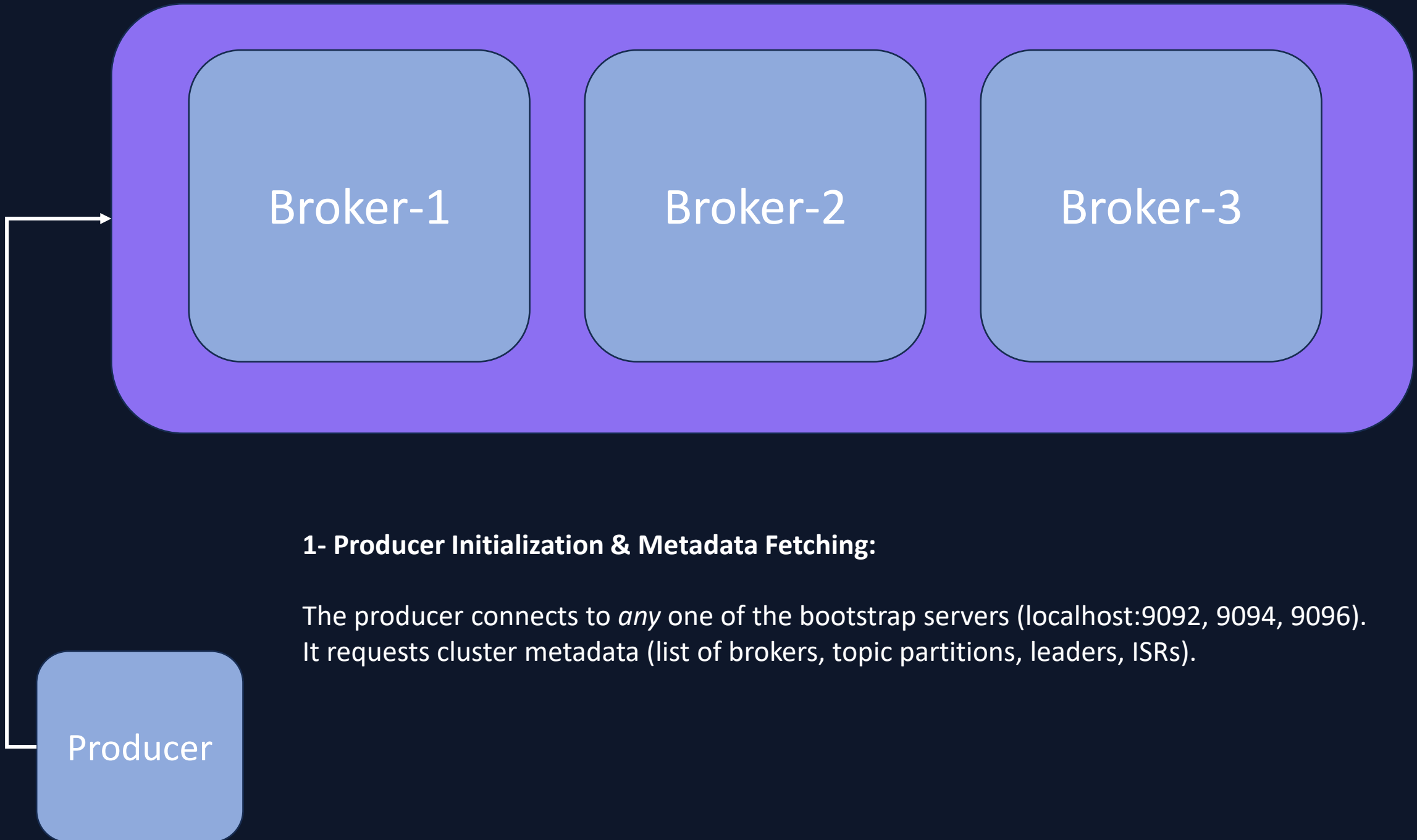
# Why Clustering?

یکی از دلایل اصلی کلاسترینگ در سیستم‌های عملیاتی بزرگ، بحث مدیریت **failure** است. بعنوان مثال اگر تنها یک بروکر داشته باشیم، با رخ دادن هر مشکلی برای آن یک سرور، ممکن است کل دیتای خود را از دست بدهیم. ولی هنگامی که 3 سرور داریم و **replication factor** را روی 3 تنظیم می‌کنیم، دیتای ما روی 3 سرور ذخیره می‌شود. فرایند مدیریت این امر نیز به عهده‌ی کافکاست. پس حتی اگر 2 سرور به هر دلیلی از شبکه خارج شوند، کماکان یک نود وجود دارد که دیتاهای ما اونجا کپی شده باشد.

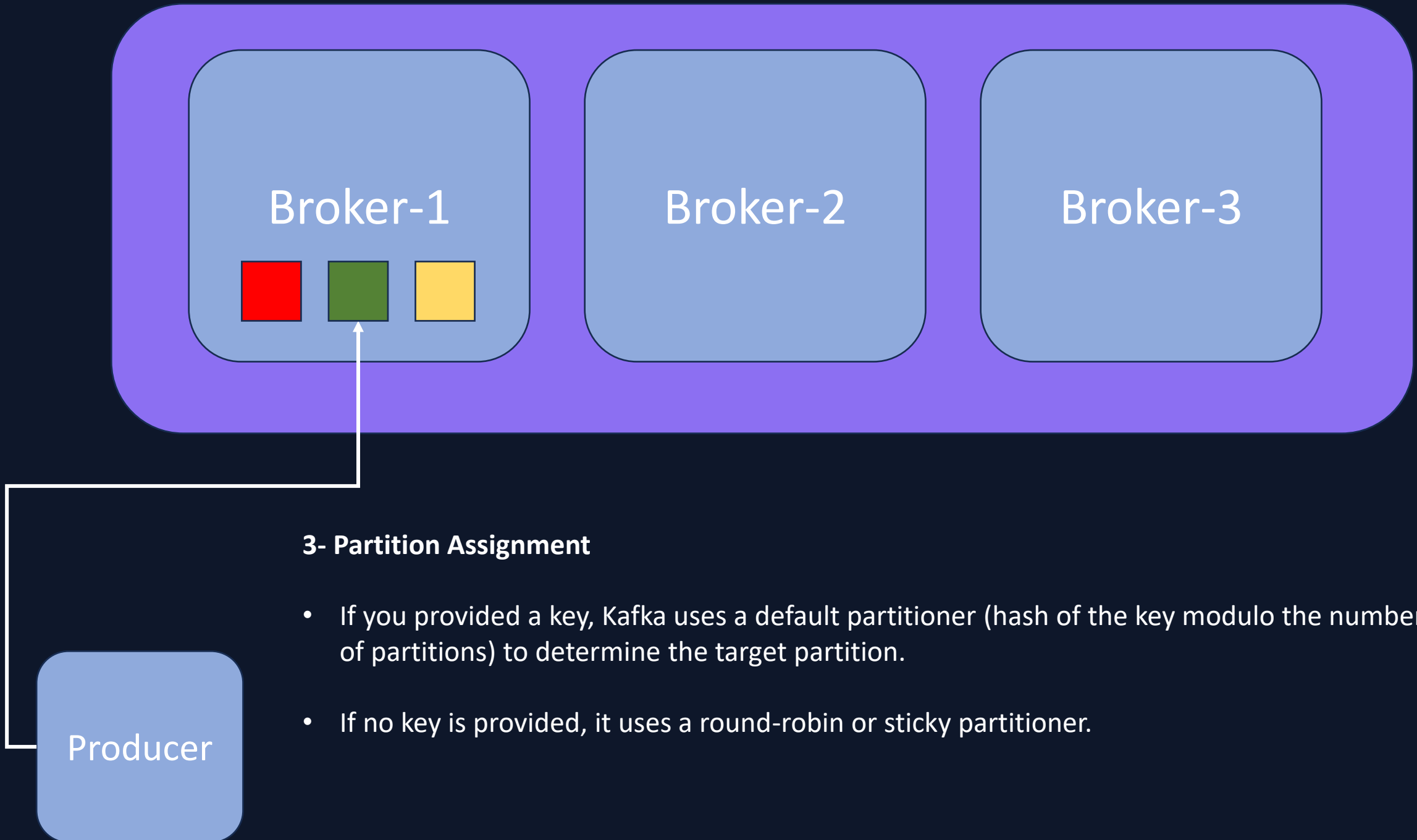


# Storing system of Kafka

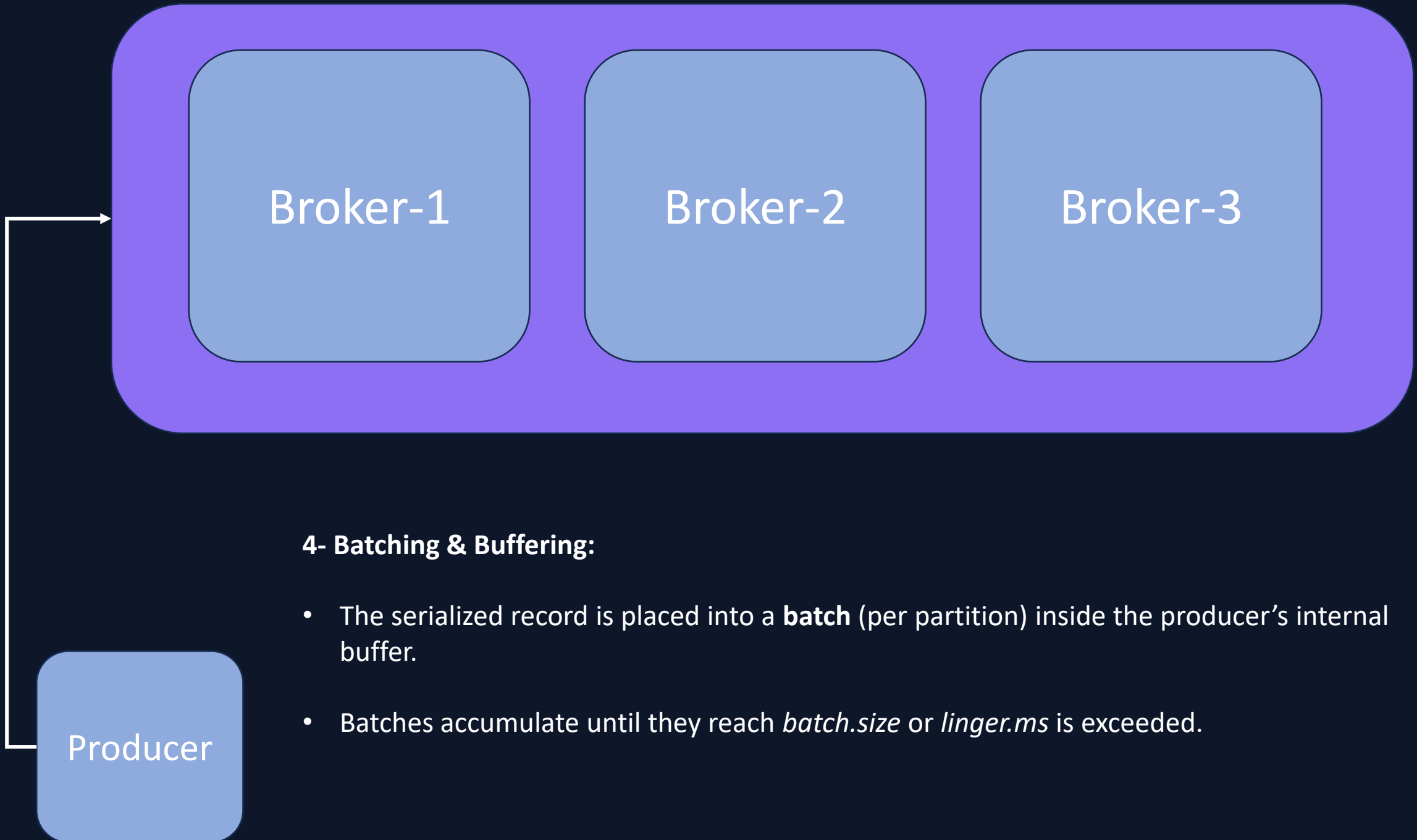


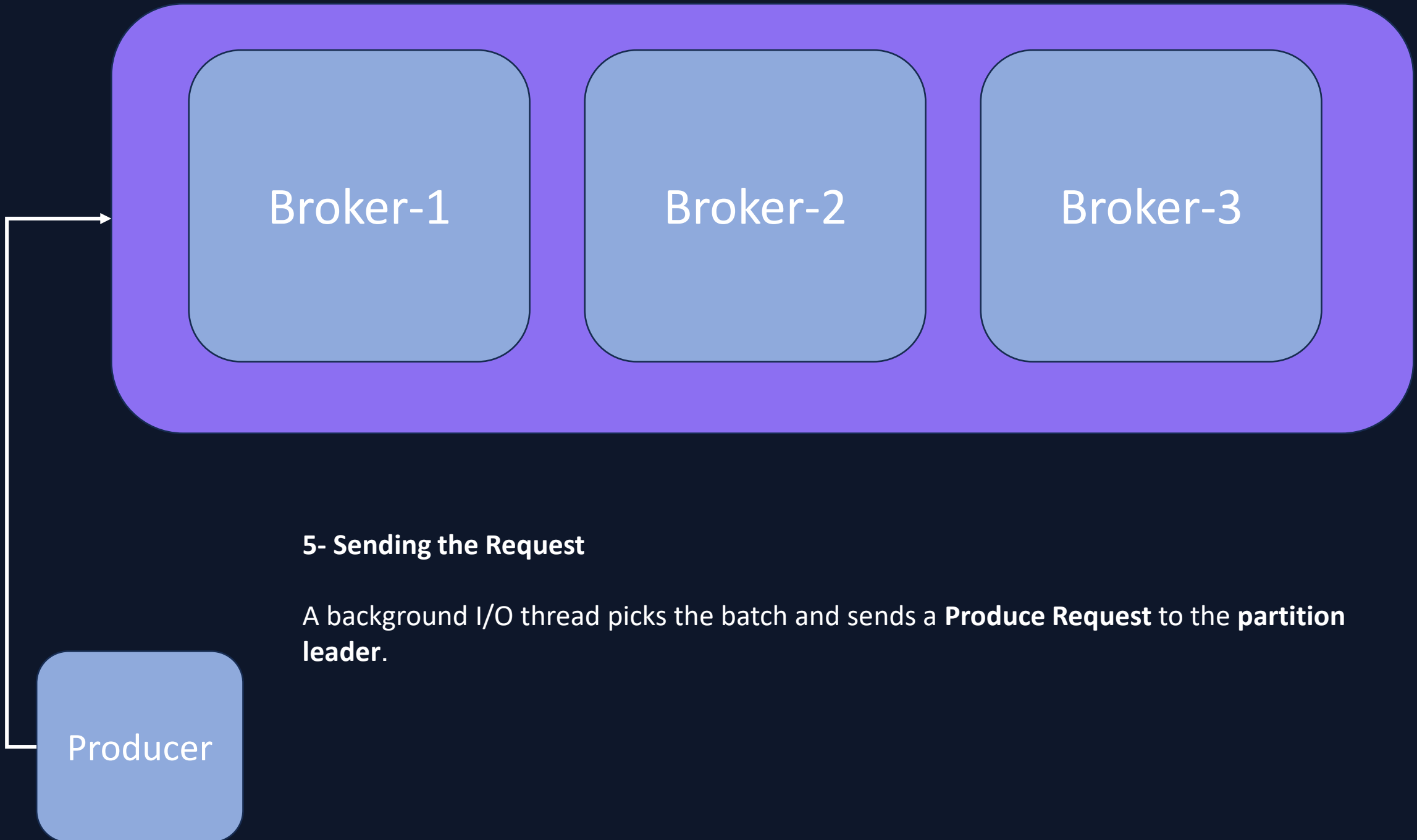


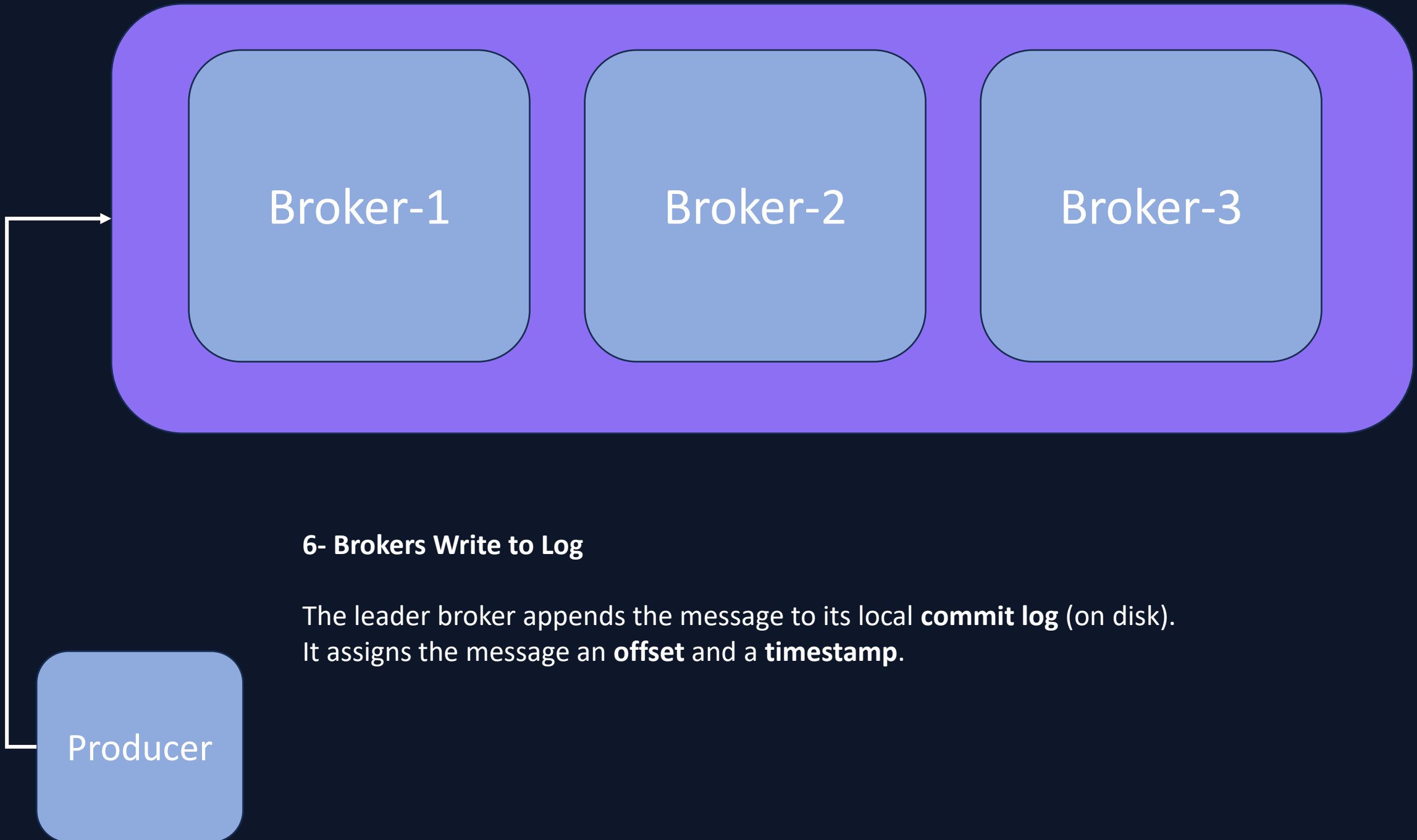


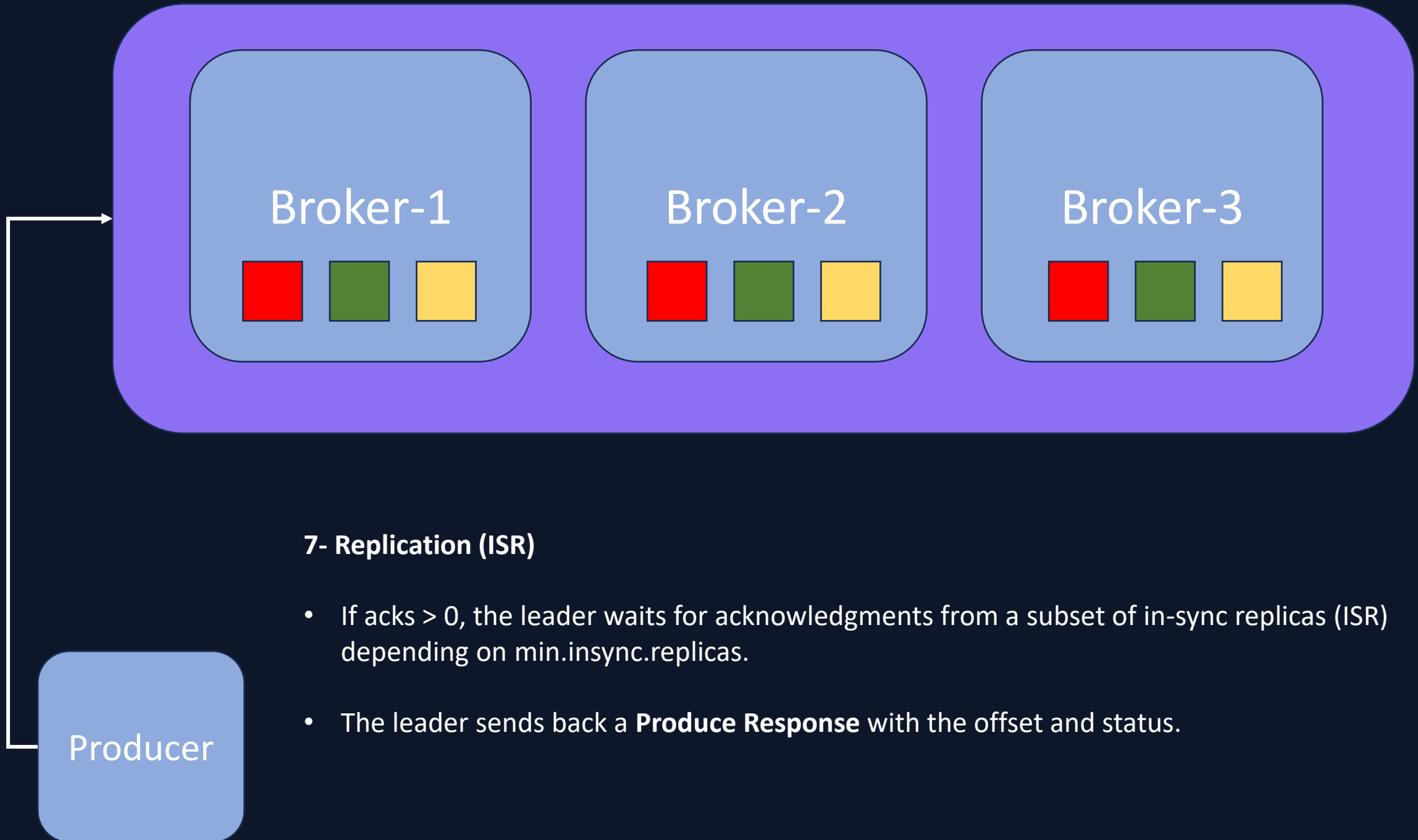


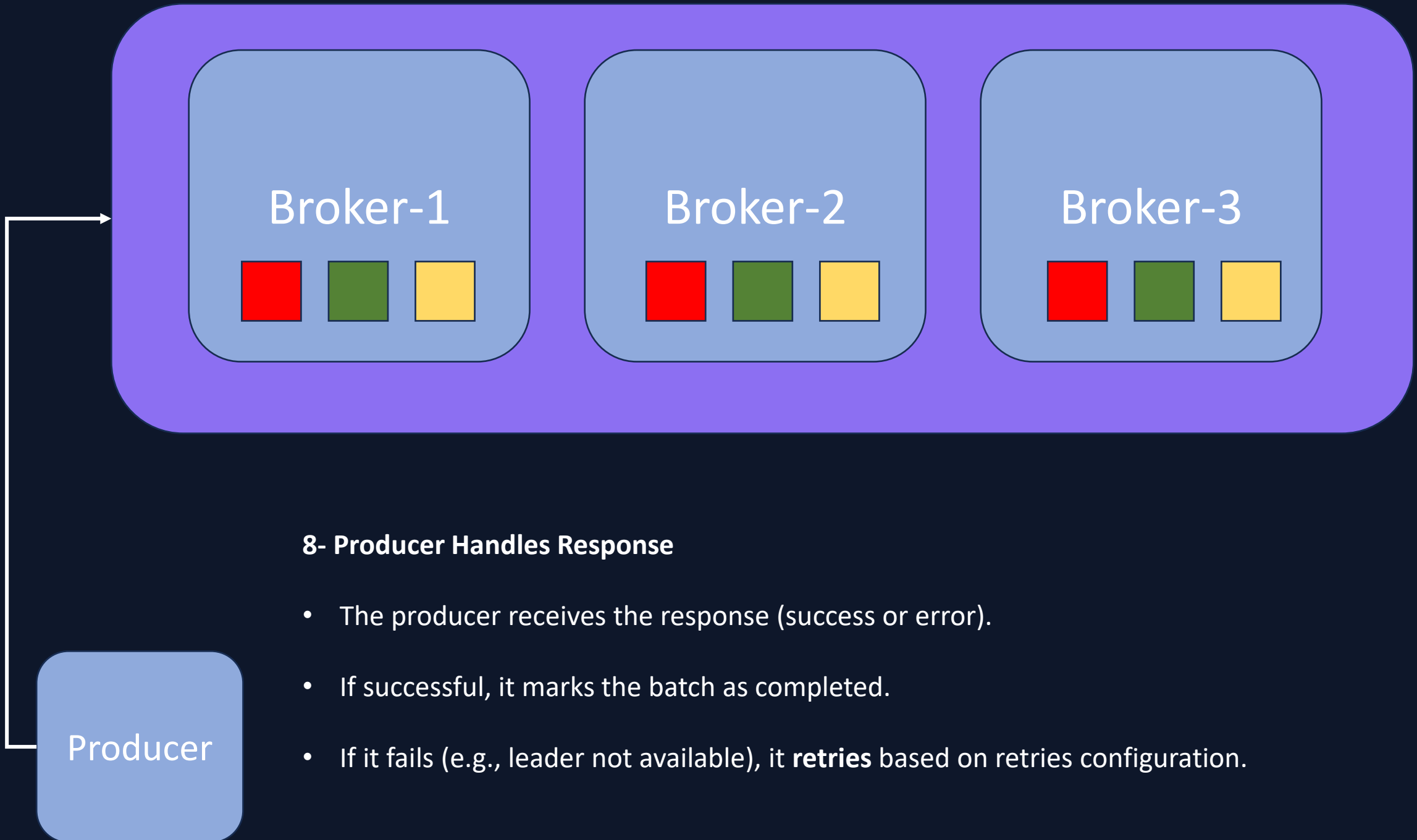




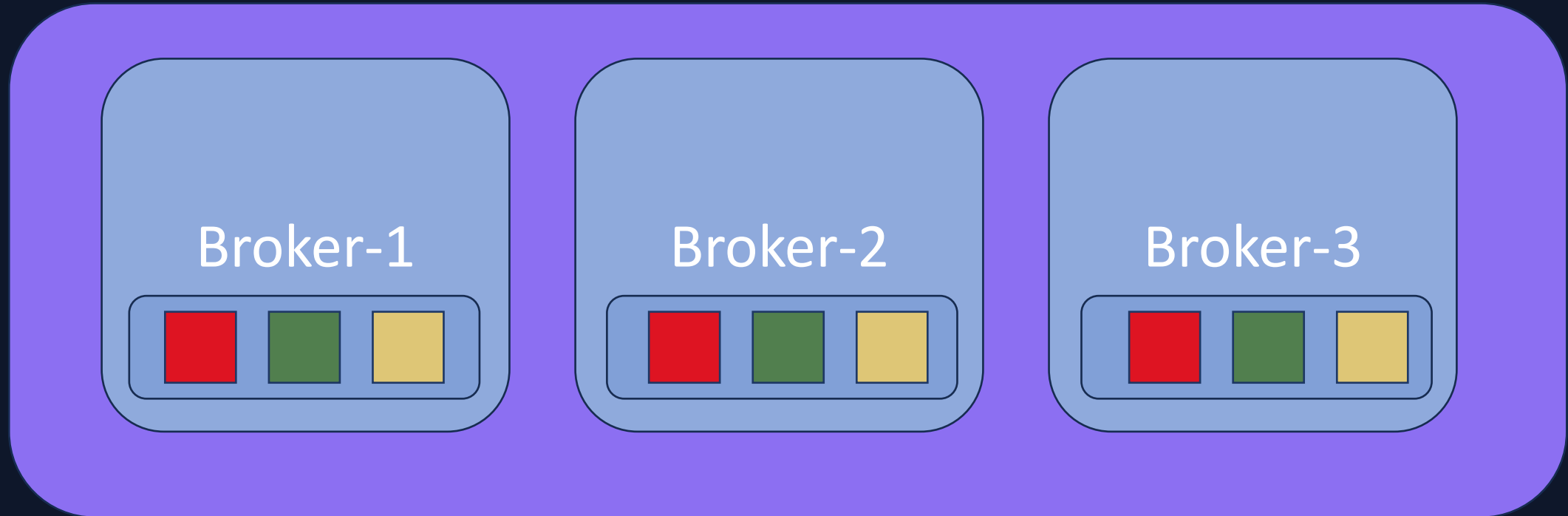








Topic X has 3 partitions and replication factor 3



Leader  
Follower  
Follower

Follower  
Follower  
Leader

Follower  
Leader  
Follower

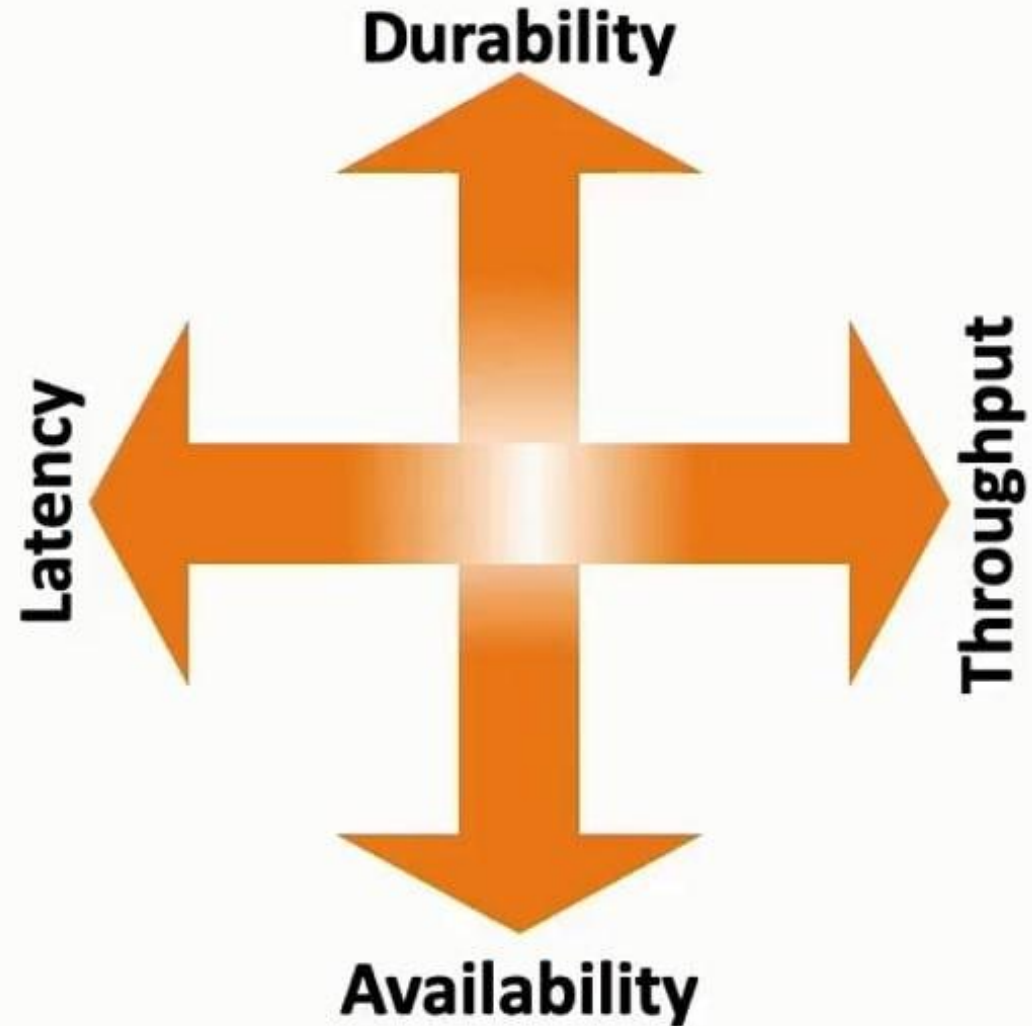
# Golden Ratio in Kafka Clustering



A production grade Kafka cluster is spread into 3 availability zones with a replication factor of 3 and minimum in-sync replicas of 2

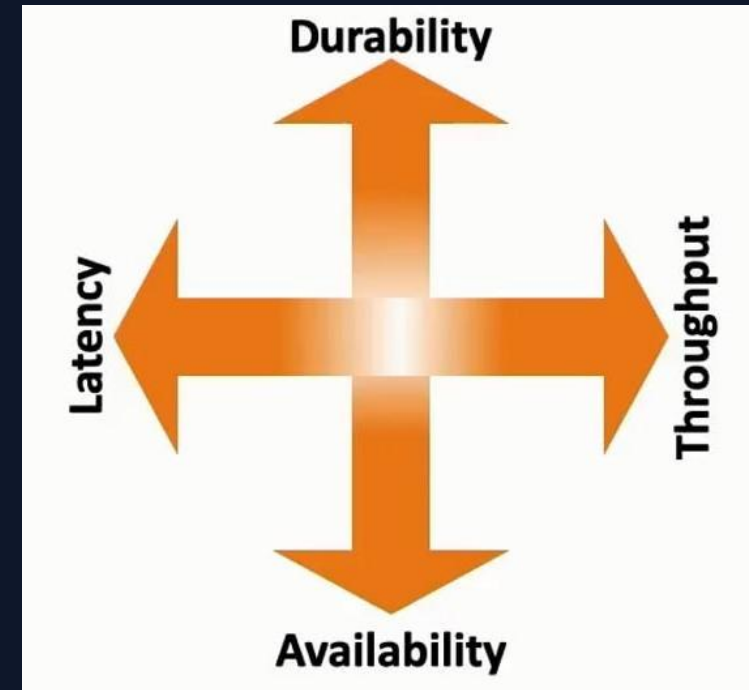


# Trade-Offs, Always



# Durability vs. Availability

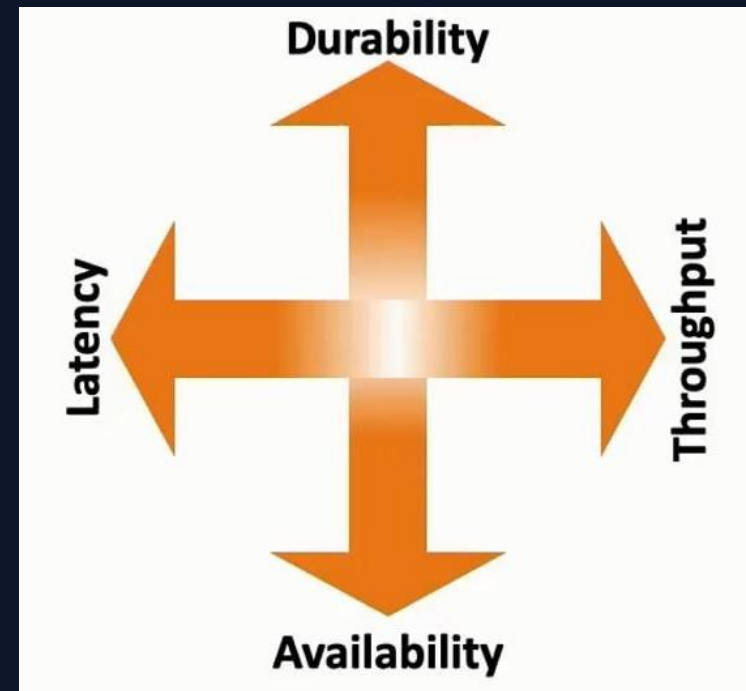
- **Durability** ensures that messages are not lost and are persisted to disk. This typically requires **writing data to multiple replicas**.
- **Availability** means that the system can continue to accept and serve requests even when some replicas are down. This often requires sacrificing strict consistency or durability guarantees, such as allowing writes to succeed even if not all replicas are up-to-date.



# Throughput vs. Latency

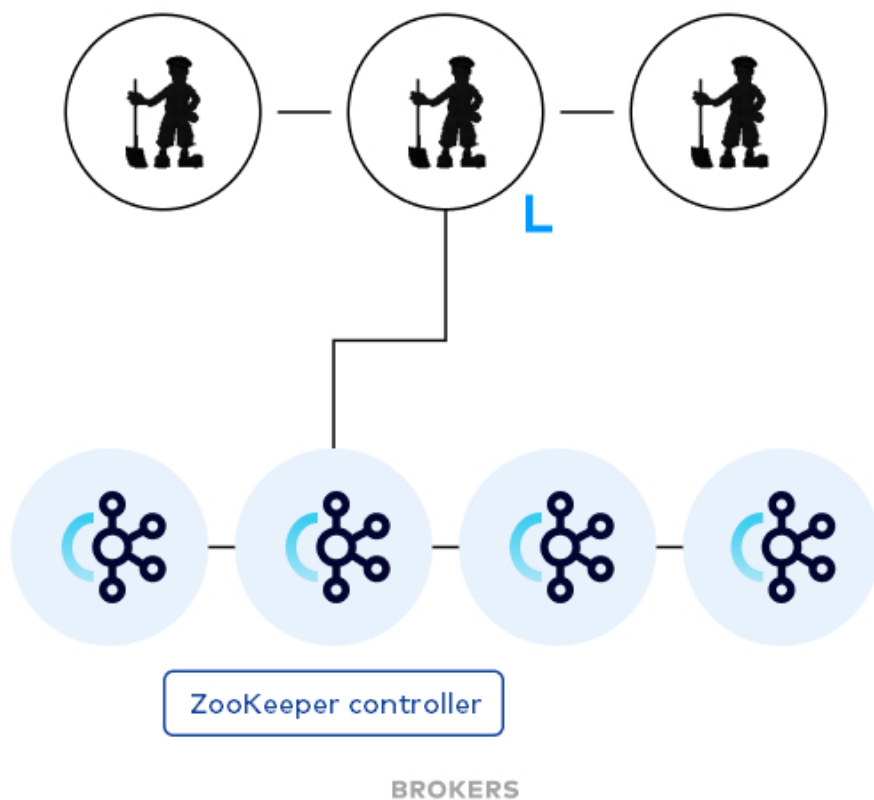
- **Throughput** refers to the number of messages processed in a given time period. To optimize throughput, systems often batch messages and process them in larger chunks.
- **Latency** is the time taken to process a single message. Lowering latency often means processing messages individually and immediately, which can reduce the overall throughput.

Therefore, maximizing throughput can lead to higher latency since the system may hold messages in batches before processing them. Conversely, focusing on low latency might mean processing messages one at a time, thereby reducing overall throughput.

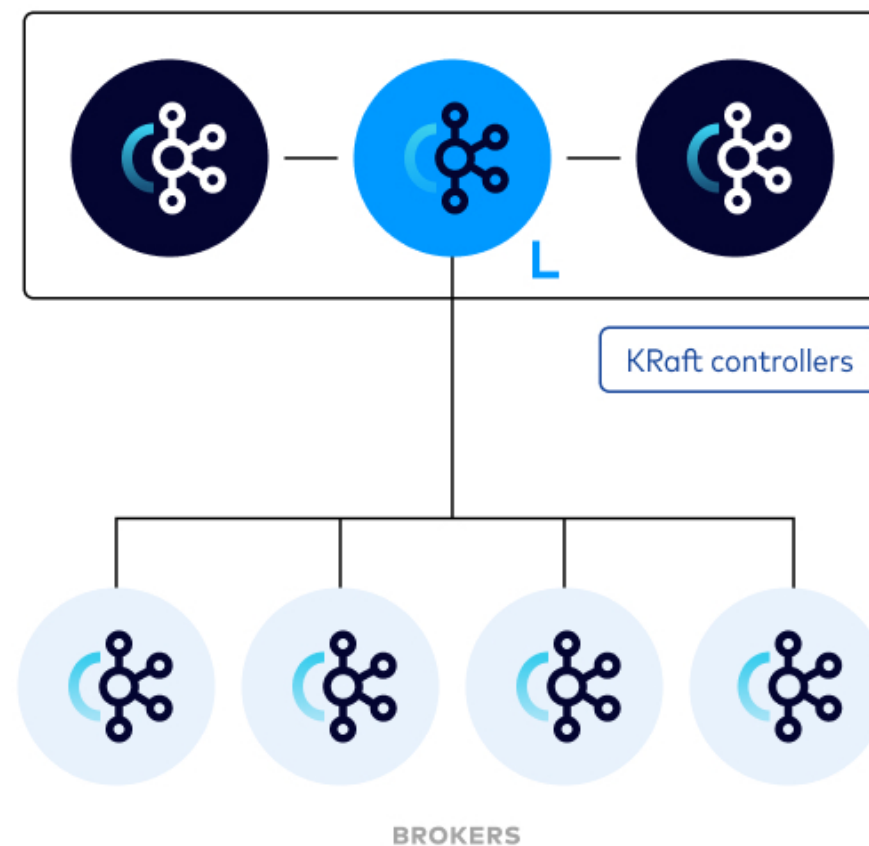


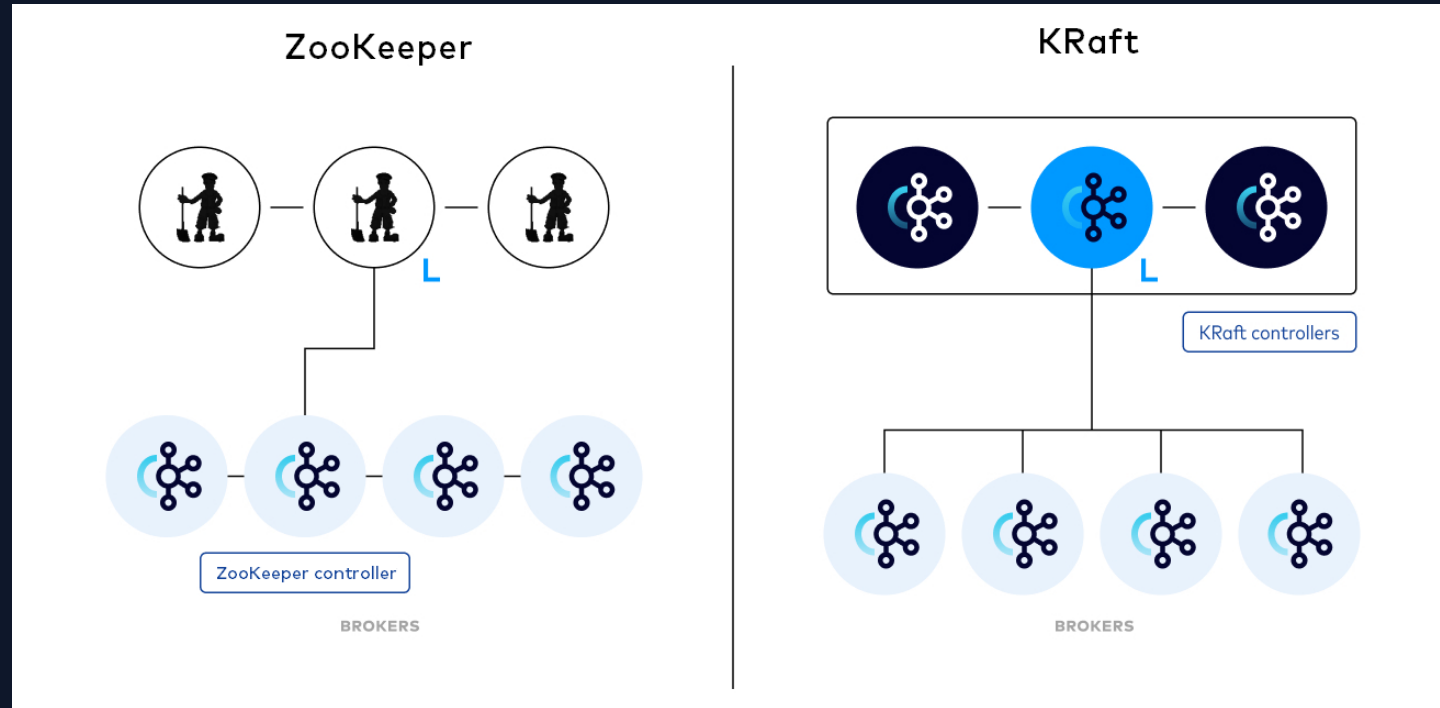
# Zookeeper vs. KRaft

ZooKeeper



KRaft





**Controller in KRaft Mode** (Since Kafka 3.3.1): With KRaft (Apache Kafka® Raft), the controller functionality is distributed among a subset of brokers designated as controllers. These controllers form a quorum and use the Raft consensus protocol to agree on cluster metadata changes and elect a leader for cluster. **The leader controller is responsible for handling requests from brokers and clients, such as creating topics, assigning partitions, and changing configurations.**

# بریم برای انجام کار عملی کلاسترینگ کافکا

